

1 Introduction

The technique used in the proofs of the existence of uncomputable functions and of the undecidability of *HALT* [AB09, Theorems 1.10 and 1.11] is called *diagonalization*. It is essentially the only general technique we know for proving that two complexity classes are *different*: to separate a class $\mathcal{C}$ from a larger class, we must exhibit a machine in the larger class whose answer differs from that of every machine in $\mathcal{C}$ on at least one input.

Recall the setup from [AB09, Section 1.4]. Every string $x \in \{0, 1\}^*$ represents a Turing Machine, denoted by M_x , and every TM is represented by infinitely many strings. The generic diagonalization argument constructs a machine D that, on input x , outputs $1 - M_x(x)$ whenever M_x belongs to $\mathcal{C}$ (the behavior of D when M_x does not halt has to be defined with some care). Obtaining the value $1 - M_x(x)$ is not difficult: D simply runs the universal TM U of [AB09, Theorem 1.9] on (x, x) and flips the answer. By construction, D disagrees with every machine $M_x \in \mathcal{C}$ on the input x , i.e., on a description of M_x itself; hence $D \notin \mathcal{C}$. In the previous lecture, this argument showed that some functions are not computable by any TM.

In this lecture, we use diagonalization to prove the Time Hierarchy Theorem and Ladner's Theorem. We then introduce *oracle Turing Machines* and use them to show that any argument relying only on two properties of TMs (TMs can be represented as strings and that a universal TM exists) cannot resolve whether $\mathbf{P} = \mathbf{NP}$. These notes follow [AB09, Chapter 3].

2 Time Hierarchy Theorem

Recall that for a function $f : \mathbb{N} \rightarrow \mathbb{N}$, $\mathbf{DTIME}(f(n))$ is the set of languages that are decided by some TM running in time $O(f(n))$ [AB09, Section 1.6]. The Time Hierarchy Theorem describes the containment relations among different $\mathbf{DTIME}$ classes: giving a TM slightly more time strictly enlarges the set of languages it can decide. As usual, we restrict attention to time-constructible functions.

Theorem 1 (Time Hierarchy Theorem [HS65]; see also [AB09, Theorem 3.1]). *If f, g are time-constructible functions satisfying $f(n) \log f(n) = o(g(n))$, then $\mathbf{DTIME}(f(n)) \subsetneq \mathbf{DTIME}(g(n))$.*

Proof Idea. Where could a language in $\mathbf{DTIME}(g(n))$ but not in $\mathbf{DTIME}(f(n))$ come from? The plan is the same as for the halting problem: build a machine D that runs in time $g(n)$ and that, on input x , deliberately gives the opposite answer to $M_x(x)$. Then D disagrees with every machine M_x on the input x , so no machine M_x that runs in time $f(n)$ can decide the same language as D .

The only question is whether D can afford to find out what $M_x(x)$ is. By [AB09, Theorem 1.9], the universal TM U simulates any T -step computation of M_x in at most $cT \log T$ steps, where c depends only on M_x . If M_x runs in time $f(n)$, the simulation costs about $c \cdot f(n) \log f(n)$ steps, and this is $o(g(n))$ by assumption. So D simply runs U on (x, x) with a budget of $g(|x|)$ steps

and flips the answer if the simulation finishes. For short descriptions x , the budget might be too small, and the constant c might be huge. But every machine has arbitrarily long descriptions, and for long enough x the budget $g(|x|)$ is enough. Hence the language of D is in $\mathbf{DTIME}(g(n))$ by construction, but not in $\mathbf{DTIME}(f(n))$.

2.1 P, NP, EXP, and NEXP

Recall that $\mathbf{EXP} = \bigcup_{c \geq 1} \mathbf{DTIME}(2^{n^c})$ and $\mathbf{NEXP} = \bigcup_{c \geq 1} \mathbf{NTIME}(2^{n^c})$. These four classes form a chain:

$$\mathbf{P} \subseteq \mathbf{NP} \subseteq \mathbf{EXP} \subseteq \mathbf{NEXP}.$$

The first and the third inclusions hold because a deterministic TM is a special case of a nondeterministic one. The second holds because a language in $\mathbf{NP}$ can be decided in exponential time by enumerating all polynomially long witnesses and running the verifier on each.

The Time Hierarchy Theorem tells us that the two ends of the chain are strictly separated. For $\mathbf{P} \subsetneq \mathbf{EXP}$, take $f(n) = 2^{\sqrt{n}}$ and $g(n) = 2^n$: then $f(n) \log f(n) = \sqrt{n} 2^{\sqrt{n}} = o(2^n)$, so $\mathbf{DTIME}(2^{\sqrt{n}}) \subsetneq \mathbf{DTIME}(2^n)$. Since $n^c = o(2^{\sqrt{n}})$ for every c , we have $\mathbf{P} \subseteq \mathbf{DTIME}(2^{\sqrt{n}}) \subsetneq \mathbf{DTIME}(2^n) \subseteq \mathbf{EXP}$. In the same way, the Nondeterministic Time Hierarchy Theorem [AB09, Theorem 3.2] gives $\mathbf{NP} \subsetneq \mathbf{NEXP}$.

On the other hand, the theorem says nothing about the two middle inclusions. Since $\mathbf{P} \neq \mathbf{EXP}$, at least one of $\mathbf{P} \subseteq \mathbf{NP}$ and $\mathbf{NP} \subseteq \mathbf{EXP}$ must be strict, but we do not know which. The two questions are related by a padding argument: if $\mathbf{P} = \mathbf{NP}$, then $\mathbf{EXP} = \mathbf{NEXP}$ [AB09, Chapter 2]; equivalently, separating $\mathbf{EXP}$ from $\mathbf{NEXP}$ would separate $\mathbf{P}$ from $\mathbf{NP}$.

3 Ladner's Theorem

Among Karp's 21 problems, every $\mathbf{NP}$ problem that was not known to be in $\mathbf{P}$ turned out to be $\mathbf{NP}$ -complete, i.e., polynomial-time equivalent to 3-SAT. This suggests a natural conjecture: is every problem in $\mathbf{NP}$ either in $\mathbf{P}$ or $\mathbf{NP}$ -complete? The question is only meaningful when $\mathbf{P} \neq \mathbf{NP}$; otherwise every problem in $\mathbf{NP}$ is in $\mathbf{P}$. Ladner's Theorem answers the question negatively: assuming $\mathbf{P} \neq \mathbf{NP}$, there is an “intermediate” language that is neither in $\mathbf{P}$ nor $\mathbf{NP}$ -complete. The proof pushes the boundary of diagonalization further, via a technique called *delayed diagonalization*.

Theorem 2 (Ladner's Theorem [Lad75]; see also [AB09, Theorem 3.3]). *Suppose that $\mathbf{P} \neq \mathbf{NP}$. Then there exists a language $L \in \mathbf{NP} \setminus \mathbf{P}$ that is not $\mathbf{NP}$ -complete.*

Proof Idea. The key is to build a “continuous” transition between $\mathbf{NP}$ -complete and $\mathbf{P}$. Take $\psi \in \mathbf{SAT}$ and make the string longer by padding. For a function $H : \mathbb{N} \rightarrow \mathbb{N}$, define the language

$$\mathbf{SAT}_H = \{ \psi 01^{n^{H(n)}} : \psi \in \mathbf{SAT} \text{ and } n = |\psi| \}.$$

The amount of padding determines the difficulty of $\mathbf{SAT}_H$. If the padding is only polynomially long (say $H(n) \leq C$ for a constant C), then $\psi \mapsto \psi 01^{n^{H(n)}}$ is itself a polynomial-time reduction, and $\mathbf{SAT}_H$ remains $\mathbf{NP}$ -complete. If the padding is exponentially long (say 2^n), then the input length is $N \approx 2^n$, so enumerating all assignments of ψ takes only $O(N)$ time and $\mathbf{SAT}_H$ falls into $\mathbf{P}$. As H grows, $\mathbf{SAT}_H$ slides from $\mathbf{NP}$ -complete all the way down to $\mathbf{P}$. The question is whether we can find an H such that $\mathbf{SAT}_H$ gets stuck in the middle: neither $\mathbf{NP}$ -complete nor in $\mathbf{P}$.

Our strategy is to let H “adjust itself.” Concretely, we design H as a detector: the value $H(n)$ is determined by whether one of the first few polynomial-time TMs correctly decides SAT_H on all short inputs. As long as some machine has not made a mistake, H stays at the index of that machine; once the machine is caught making a mistake, H continues to grow. The resulting H satisfies a key property:

$$\text{SAT}_H \in \mathbf{P} \iff H(n) \leq C \text{ for some constant } C.$$

That is, whether H diverges is determined exactly by the question “can SAT_H be decided in polynomial time,” which we cannot answer directly.

With this property, both directions of the argument go through. First, suppose $\text{SAT}_H \in \mathbf{P}$. Then H is bounded, so SAT_H is just SAT with polynomial padding, $\text{SAT} \leq_p \text{SAT}_H$, and hence $\text{SAT} \in \mathbf{P}$, contradicting $\mathbf{P} \neq \mathbf{NP}$. Therefore $\text{SAT}_H \notin \mathbf{P}$, and by the key property, $H(n)$ diverges.

Next, suppose SAT_H is $\mathbf{NP}$ -complete. This assumption hands us a polynomial-time reduction f satisfying $\psi \in \text{SAT} \iff f(\psi) \in \text{SAT}_H$, together with a running-time bound cn^i , where c, i are constants that come with f ; we know nothing about them, but they are fixed. On the other hand, we already know that H diverges. Note that the definition of H does not involve f at all; H was fixed before f entered the picture. Divergence, however, means that H eventually exceeds any given constant, so there is n_0 such that $H(m) > 3i$ for all $m \geq n_0$ (the factor 3 is arbitrary; anything larger than 1 works).

Now examine the output of f on an input ψ of length n . Since f runs for only cn^i steps, $|f(\psi)| \leq cn^i$. If $f(\psi)$ is not a legal SAT_H instance (i.e., not of the form $\varphi 01^{m^{H(m)}}$ with $m = |\varphi|$), then the correctness of the reduction immediately gives $\psi \notin \text{SAT}$. If it is a legal instance $f(\psi) = \varphi 01^{m^{H(m)}}$, then on one hand $\psi \in \text{SAT} \iff \varphi \in \text{SAT}$, and on the other hand the padding must fit inside the output:

$$m^{H(m)} \leq |f(\psi)| \leq cn^i.$$

When $m \geq n_0$, the left-hand side is at least m^{3i} , so $m^{3i} \leq cn^i$, i.e., $m \leq c^{1/(3i)} \cdot n^{1/3}$; when $m < n_0$, m is a constant anyway. That is, the length of φ is at most of the order $\sqrt[3]{n}$: without knowing anything, the reduction f compresses ψ into a much shorter equivalent instance. The source of the compression is that the length of a legal SAT_H instance is super-polynomial in the length of its inner formula (the exponent $H(m)$ is unbounded), whereas the output length of f is only polynomial in n (the exponent i is fixed). When the exponent is unbounded, the base must be tiny.

Thus, in either case, deciding ψ reduces to deciding an instance of size at most $\sqrt[3]{n}$. Repeating this, each round takes polynomial time, and the size shrinks by a cube root, so the number of rounds is at most logarithmic. This gives a polynomial-time algorithm for SAT , again contradicting $\mathbf{P} \neq \mathbf{NP}$. Therefore, under $\mathbf{P} \neq \mathbf{NP}$, SAT_H is neither in $\mathbf{P}$ nor $\mathbf{NP}$ -complete.

It is worth noting how H is constructed. Classic diagonalization (as in the Time Hierarchy Theorem) “defeats” a machine on the spot at each input. Here we cannot do so, because SAT_H must always keep the shape of “padded SAT ” and we are not allowed to flip answers arbitrarily. We can only wait: SAT_H keeps its current padding rate until a counterexample to some machine shows up by itself. We cannot predict when the counterexample appears; every step of the diagonalization is “delayed” until the evidence arrives, which is why the technique is called *delayed diagonalization*.

3.1 Proof

Due to limited class time, we were unable to finish going through the proof of Ladner's theorem. A detailed proof of the theorem is provided below.

Proof. We follow the proof idea and fill in the details. For $i \in \mathbb{N}$, let M_i denote the TM represented by the binary expansion of i , and let $\text{SAT}_H(x) = 1$ iff $x \in \text{SAT}_H$.

Definition of H . Define $H : \mathbb{N} \rightarrow \mathbb{N}$ as follows:

$H(n)$ is the smallest number $i < \log \log n$ such that for every $x \in \{0, 1\}^*$ with $|x| \leq \log n$, M_i outputs $\text{SAT}_H(x)$ within $i|x|^i$ steps. If there is no such i , then $H(n) = \log \log n$.

A few words on the three parameters. The condition $|x| \leq \log n$ makes $H(n)$ cheap to compute: the strings involved are so short that brute-forcing SAT on them costs only $2^{\log n} = n$ steps. The bound $i|x|^i$ is a single family of polynomial bounds that loosens as i grows; any machine running in time cn^c is represented by some index $i > c$ (in fact by infinitely many), and for that index the bound $i|x|^i$ covers it. The cap $i < \log \log n$ simply makes the search finite and forces H to grow slowly.

This definition is not circular: $H(n)$ affects the membership in SAT_H only of strings of length greater than n (a legal instance $\psi 01^{n^{H(n)}}$ with $|\psi| = n$ is longer than n), whereas computing $H(n)$ only requires membership in SAT_H of strings of length at most $\log n$. Moreover, the definition yields a polynomial-time recursive algorithm for H : to compute $H(n)$ we enumerate the $O(n)$ strings x of length at most $\log n$, decide $\text{SAT}_H(x)$ (parse $x = \psi 01^k$, compute $H(|\psi|)$ recursively with $|\psi| < \log n$, check $k = |\psi|^{H(|\psi|)}$, and decide $\psi \in \text{SAT}$ by brute force in $2^{|\psi|} \text{poly}(|\psi|) \leq \text{poly}(n)$ time), and simulate each M_i , $i < \log \log n$, for $i|x|^i \leq (\log \log n)(\log n)^{\log \log n} = n^{o(1)}$ steps. Memoizing the recursive calls, the whole computation takes polynomial time; see [AB09, Exercise 3.6].

$\text{SAT}_H \in \text{NP}$. On input y , a nondeterministic polynomial-time machine guesses ψ and an assignment to ψ , checks that $y = \psi 01^{n^{H(n)}}$ with $n = |\psi|$ (this uses that H is polynomial-time computable), and checks that the assignment satisfies ψ .

Key Claim. $\text{SAT}_H \in \text{P}$ iff $H(n) = O(1)$, i.e., there is a constant C such that $H(n) \leq C$ for all n . Moreover, if $\text{SAT}_H \notin \text{P}$, then $H(n)$ tends to infinity with n .

Proof of the Claim. See [AB09, Theorem 3.3]. □

$\text{SAT}_H \notin \text{P}$. Suppose $\text{SAT}_H \in \text{P}$. By the Claim, $H(n) \leq C$ for some constant C , so the map $\psi \mapsto \psi 01^{|\psi|^{H(|\psi|)}}$ is computable in polynomial time (as H is) and produces an output of length at most $|\psi|^C + |\psi| + 1$. This map is a polynomial-time reduction from SAT to SAT_H , so a polynomial-time algorithm for SAT_H yields one for SAT , implying $\text{P} = \text{NP}$ and contradicting the assumption. Hence $\text{SAT}_H \notin \text{P}$, and by the Claim, $H(n) \rightarrow \infty$.

SAT_H is not NP-complete. Suppose SAT_H is NP-complete, so there is a reduction f from SAT to SAT_H running in time cn^i for constants c, i . Since $H(n) \rightarrow \infty$, fix n_0 such that $H(m) > 3i$ for all $m \geq n_0$. (Here we need H to *diverge*, not merely to be unbounded: we want $H(m)$ to be large for *every* large m , which is exactly the “moreover” part of the Claim.) As computed in the proof idea, for every ψ with $|\psi| = n$, one of the following holds: (a) $f(\psi)$ is not of the form $\varphi 01^{m^{H(m)}}$

with $m = |\varphi|$, in which case $\psi \notin \text{SAT}$; or (b) $f(\psi) = \varphi 01^{m^{H(m)}}$ with $m = |\varphi|$, in which case $\psi \in \text{SAT} \iff \varphi \in \text{SAT}$ and $m \leq \max\{n_0, c^{1/(3i)}n^{1/3}\}$. Consider the following recursive algorithm A for SAT :

“On input ψ of length n : if $n < n_1$ (for a constant $n_1 \geq n_0$ chosen so that $c^{1/(3i)}n^{1/3} < n$ for all $n \geq n_1$), decide $\psi \in \text{SAT}$ by brute force. Otherwise, compute $y = f(\psi)$ and check, using the polynomial-time algorithm for H , whether $y = \varphi 01^{m^{H(m)}}$ for some φ with $m = |\varphi|$. If not, reject. If so, output $A(\varphi)$.”

Correctness follows from (a) and (b) by induction on n . For the running time, each recursive level costs $\text{poly}(n)$ (computing f , computing $H(m)$, and parsing), and the input size shrinks from n to at most $c^{1/(3i)}n^{1/3} < n$ in each level, so after $O(\log \log n)$ levels the size drops below n_1 . Hence A runs in polynomial time and $\text{SAT} \in \mathbf{P}$, implying $\mathbf{P} = \mathbf{NP}$ and contradicting the assumption. (This is [AB09, Exercise 3.6(b)].)

Therefore, assuming $\mathbf{P} \neq \mathbf{NP}$, the language $L = \text{SAT}_H$ is in $\mathbf{NP} \setminus \mathbf{P}$ and is not $\mathbf{NP}$ -complete. $\square$

4 Oracle Turing Machines

So far we have seen what diagonalization can prove. The technique, however, has its limits: it cannot resolve whether $\mathbf{P} = \mathbf{NP}$ by relying only on the two properties that TMs can be represented as strings and that a universal TM exists. To make this precise, we introduce a new concept: *oracle Turing Machines*.

4.1 Definition

An oracle Turing Machine is a TM that is given access to a black box, or “oracle,” that can magically decide membership in some language $O \subseteq \{0, 1\}^*$.

Definition 3 (Oracle Turing Machines [AB09, Definition 3.4]). An *oracle Turing Machine* is a TM M that has a special read-write tape, called the *oracle tape*, and three special states $q_{\text{query}}, q_{\text{yes}}, q_{\text{no}}$. To execute M , we specify, in addition to the input, a language $O \subseteq \{0, 1\}^*$ used as the oracle. Whenever M enters the state q_{query} , it moves to the state q_{yes} if $q \in O$ and to q_{no} if $q \notin O$, where q denotes the current content of the oracle tape. Regardless of O , a query counts as a single computational step. We denote by $M^O(x)$ the output of M on input x with oracle O . Nondeterministic oracle TMs are defined similarly.

Definition 4 ([AB09, Definition 3.5]). For every $O \subseteq \{0, 1\}^*$, $\mathbf{P}^O$ is the set of languages that can be decided by a polynomial-time deterministic TM with oracle access to O , and $\mathbf{NP}^O$ is the set of languages that can be decided by a polynomial-time nondeterministic TM with oracle access to O .

4.2 What it tells us

The two properties above (representation as strings, existence of a universal TM) also hold in the oracle world: oracle TMs can be represented as strings, and a universal TM that itself has access to the oracle O can simulate them. Therefore, any theorem about the non-oracle world that is proved using only these two properties can be transported to the oracle world, for every oracle O . But in the oracle world, the answer to whether $\mathbf{P}$ equals $\mathbf{NP}$ differs depending on the choice of the oracle A or B . Hence, in the non-oracle world, the question of whether $\mathbf{P} = \mathbf{NP}$ cannot be settled using only these two properties: if such a proof existed, then for any oracles A and B , the answer to

whether $\mathbf{P}^A = \mathbf{NP}^A$ and whether $\mathbf{P}^B = \mathbf{NP}^B$ would have to be the same. The following theorem tells us that this is not the case.

Theorem 5 (Baker, Gill, Solovay [BGS75]; see also [AB09, Theorem 3.7]). *There exist oracles A , B such that $\mathbf{P}^A = \mathbf{NP}^A$ and $\mathbf{P}^B \neq \mathbf{NP}^B$.*

References

- [AB09] Sanjeev Arora and Boaz Barak. *Computational Complexity: A Modern Approach*. Cambridge University Press, 2009.
- [BGS75] Theodore Baker, John Gill, and Robert Solovay. Relativizations of the $\mathbf{P}=?\mathbf{NP}$ question. *SIAM Journal on computing*, 4(4):431–442, 1975.
- [HS65] Juris Hartmanis and Richard E Stearns. On the computational complexity of algorithms. *Transactions of the American Mathematical Society*, 117:285–306, 1965.
- [Lad75] Richard E Ladner. On the structure of polynomial time reducibility. *Journal of the ACM (JACM)*, 22(1):155–171, 1975.